

1. HOW INTERNET & WEB ACTUALLY WORK

What happens when you open a website?

Example:

`https://google.com`

STEP 1 — Browser Reads URL

URL has:

Protocol + Domain

Example:

`https://google.com`

- `https` → communication rule
 - `google.com` → website name
-

STEP 2 — DNS

Humans remember names.

Computers use IP addresses.

DNS converts:

`google.com` → `142.x.x.x`

WHY?

Because servers communicate using IP.

DNS = Internet phonebook.

STEP 3 — TCP CONNECTION

Browser creates reliable connection.

WHY TCP?

Because web pages cannot lose data.

TCP guarantees:

- order
 - delivery
 - reliability
-

STEP 4 — HTTPS SECURITY

HTTPS = HTTP + SSL/TLS encryption.

WHY encryption?

To protect:

- passwords
- banking
- private data

Without HTTPS:

Data can be intercepted.

STEP 5 — HTTP REQUEST

Browser asks server:

GET /index.html

Meaning:

“Give me webpage.”

STEP 6 — SERVER RESPONSE

Server sends:

- HTML
 - CSS
 - JS
 - Images
-

STEP 7 — DOM CREATION

Browser converts HTML into:

DOM Tree

WHY DOM?

Because JavaScript needs a structure to manipulate webpage.

CORE WEB UNDERSTANDING

HTML = structure

CSS = design

JS = behavior

Browser combines all 3.

2. HTTP METHODS (VERY IMPORTANT)

GET

Purpose:

Fetch data

WHY safe?

Because it should NOT modify server data.

WHY idempotent?

Multiple GET requests give same result.

Example:

Open profile page

POST

Purpose:

Create new data

Example:

- signup
- login
- form submission

WHY not idempotent?

Submitting form twice may create duplicate entries.

PUT

Purpose:

Full update

WHY idempotent?

Same update repeated → same final result.

PATCH

Purpose:

Partial update

Example:

Only update username.

DELETE

Purpose:

Delete resource

SAFE vs IDEMPOTENT

SAFE

Does NOT change data.

Example:

GET

IDEMPOTENT

Repeating request gives same result.

Examples:

GET, PUT, DELETE

3. HTTP STATUS CODES

200

Success

Request worked.

201

Created

New resource created successfully.

301

Permanent Redirect

Page moved permanently.

304

Not Modified

Browser can use cached version.

WHY useful?

Faster loading.

400

Bad Request

Client sent invalid data.

401

Unauthorized

Authentication required.

403

Forbidden

Server understood request BUT denies access.

404

Page Not Found

Requested resource missing.

500

Internal Server Error

Problem on server side.

4. HTML — REAL UNDERSTANDING

HTML defines:

- structure
- meaning
- semantics

NOT just “tags.”

WHY SEMANTIC TAGS EXIST

Example:

```
<nav>
<header>
<footer>
<section>
<article>
```

WHY?

Because browser/search engine understands meaning.

Benefits:

- SEO
 - accessibility
 - screen readers
-

IMPORTANT TAGS

<a>

Creates hyperlinks.

WHY?

To navigate pages.

Displays image.

Important attribute:

alt=""

WHY alt exists?

Accessibility + image fallback.

<form>

Collects user data.

IMPORTANT FORM ATTRIBUTES

action

Where data goes.

method

How data travels.

GET:

- visible in URL

POST:

- hidden in body
-

readonly vs disabled

readonly

Cannot edit.

BUT submitted.

disabled

Cannot edit.

NOT submitted.

Very important conceptual difference.

5. CSS — REAL UNDERSTANDING

CSS solves:

- layout

- spacing
 - alignment
 - responsiveness
-

BOX MODEL (MOST IMPORTANT)

Every element:

margin

border

padding

content

PADDING

Space INSIDE border.

Makes content breathe.

MARGIN

Space OUTSIDE border.

Separates elements.

WHY FLEXBOX EXISTS

Old layouts using float were difficult.

Flexbox solves:

- alignment
 - spacing
 - responsiveness
-

FLEXBOX CORE IDEA

`display:flex;`

Children become flexible items.

justify-content

Controls:

LEFT ↔ RIGHT

(main axis)

align-items

Controls:

TOP ⇕ BOTTOM

(cross axis)

GRID

2D layout system.

Handles:

- rows
- columns

Better for full page layouts.

POSITION TYPES

static

Default flow.

relative

Moves relative to itself.

absolute

Moves relative to nearest positioned parent.

fixed

Fixed to screen.

Example:

sticky navbar.

sticky

Acts normal THEN sticks during scroll.

z-index

Controls stacking order.

Higher value appears on top.

6. JAVASCRIPT — MOST IMPORTANT SECTION

JavaScript controls:

- logic
 - DOM
 - interaction
 - async behavior
-

DATA TYPES

Primitive

string

number

boolean

null
undefined
symbol
bigint

Stored directly.

Non-Primitive

object
array
function

Stored by reference.

WHY ARRAY IS OBJECT?

```
typeof []  
// object
```

Because arrays internally behave like objects.

WHY typeof null === "object" ?

Old JavaScript bug.
Still exists.

Very famous question.

VARIABLES

var

Function-scoped.

Hoisted with:

undefined

let

Block-scoped.

Temporal Dead Zone.

const

Cannot reassign reference.

HOISTING

Declarations move upward during memory phase.

WHY THIS WORKS

```
console.log(a);
```

```
var a = 10;
```

Output:

undefined

Because:

```
var a;
```

exists before execution.

WHY THIS FAILS

```
console.log(a);
```

```
let a = 10;
```

Because let exists in TDZ.

== vs ===

===

Checks:

- value
 - datatype
-

==

Performs type conversion.

WHY:

5 == "5"

TRUE?

Because string converted to number.

TYPE COERCION RULES

MOST IMPORTANT.

+ prefers STRING

2 + "2"

// "22"

Concatenation.

- * / prefer NUMBER

"5" - "2"

// 3

String converted to number.

TRUTHY & FALSY

ONLY THESE ARE FALSE:

false

0

""

null

undefined

NaN

Everything else TRUE.

IMPORTANT

```
Boolean("false")
```

```
// true
```

WHY?

Because non-empty string.

DOUBLE NOT

```
!!value
```

Converts into boolean.

WHY [] == false

Flow:

```
[] -> ""
```

```
"" -> 0
```

```
false -> 0
```

Then:

```
0 == 0
```

TRUE.

WHY [] == [] IS FALSE

Objects compare references.

Different memory locations.

ARRAYS

push()

Add at end.

pop()

Remove from end.

shift()

Remove first.

unshift()

Add first.

map()

Transforms array.

filter()

Keeps matching elements.

reduce()

Accumulates values.

DOM

DOM = webpage structure in browser memory.

WHY DOM?

JS needs a way to manipulate webpage.

IMPORTANT METHODS

`getElementById()`

`querySelector()`

`querySelectorAll()`

CONTENT CHANGE

`innerHTML`

`textContent`

`innerText`

EVENTS

Examples:

`onclick`

`onchange`

`oninput`

`onsubmit`

EVENT BUBBLING

Events move:

child → parent

Stop using:

`stopPropagation()`

`preventDefault()`

Stops browser default behavior.

Example:

prevent form submission.

7. ASYNCHRONOUS JAVASCRIPT

VERY IMPORTANT.

JavaScript is:

Single-threaded

One call stack.

THEN HOW ASYNC WORKS?

Using:

- Web APIs
 - Callback queue
 - Event loop
-

EVENT LOOP

1. Sync code executes first
 2. Async tasks wait
 3. Callback queue checked
 4. Event loop pushes callback
-

CALLBACK

Function passed into another function.

PROMISE

Represents future result.

States:

pending

fulfilled

rejected

async/await

Cleaner syntax over promises.

8. JSON

Used for data transfer.

OBJECT → JSON

JSON.stringify()

JSON → OBJECT

JSON.parse()

9. REACT — EXAM UNDERSTANDING

React solves:

Complex UI updates

COMPONENT

Reusable UI block.

PROPS

Pass data parent → child.

Read-only.

STATE

Stores changing data.

Updates UI automatically.

HOOKS

useState

Stores state.

useEffect

Handles side effects:

- API calls
- timers

useRef

DOM references.

useMemo

Performance optimization.

10. NODE.JS

Node.js = JS outside browser.

Built on V8 engine.

WHY NODE IS FAST

Uses:

Event-driven non-blocking architecture

IMPORTANT MODULES

fs

File handling.

http

Create server.

path

Path utilities.

url

URL handling.

events

Event system.

11. MOST IMPORTANT TRICKY QUESTIONS

WHY:

$0.1 + 0.2 \neq 0.3$

Because floating-point precision issue.

Binary representation limitation.

WHY:

$\text{NaN} \neq \text{NaN}$

NaN means:

“invalid numeric result.”

WHY:

`typeof NaN`

`// number`

Because NaN belongs to numeric system internally.

WHY:

Infinity

Exists?

Division by zero in JS gives mathematical infinity.

FINAL EXAM STRATEGY

When stuck:

STEP 1

Check datatype.

STEP 2

Check coercion.

STEP 3

Check operator precedence.

STEP 4

Check object references.

STEP 5

Check truthy/falsy.

MOST IMPORTANT THINGS TO MEMORIZE

```
typeof null // object
typeof [] // object
typeof NaN // number

0 == false // true
[] == false // true
null == undefined // true
null == 0 // false

"5" + 1 // "51"
"5" - 1 // 4

[] == [] // false
{} == {} // false
```

1. == vs === — REAL CLARITY

This is the MOST asked JavaScript trap.

==

Checks value after type conversion.

```
5 == "5" // true
0 == false // true
```

===

Checks value + datatype.

```
5 === "5" // false
```

Golden Rule

- Use === in real projects
- == causes confusing coercion

2. Truthy vs Falsy Values

VERY important.

Falsy values (ONLY THESE)

false

0

-0

On

""

null

undefined

NaN

Everything else is truthy.

Boolean([]) // true

Boolean({}) // true

Boolean("0") // true

That's why:

!![]

// true

3. null vs undefined

undefined

Variable exists but value not assigned.

let x;

console.log(x); // undefined

null

Intentional empty value.

let user = null;

Weird JS fact

typeof null

// object

Historical JavaScript bug.

4. map() vs filter() vs forEach()

map()

Transforms array.

```
[1,2,3].map(x => x*2)
```

Output:

```
[2,4,6]
```

filter()

Keeps matching items.

```
[1,2,3,4].filter(x => x>2)
```

Output:

```
[3,4]
```

forEach()

Only loops.

No returned array.

5. slice() vs splice()

BIG favorite.

slice()

Does NOT modify original array.

```
arr.slice(1,3)
```

splice()

Modifies original array.

Used to add/remove.

```
arr.splice(1,2)
```

6. innerHTML vs innerText vs textContent

innerHTML

Reads HTML tags too.

```
div.innerHTML = "<b>Hello</b>"
```

innerText

Only visible text.

textContent

All text, faster.

7. Event Bubbling

VERY IMPORTANT concept.

When child clicked:

```
<div>  
  <button>Click</button>  
</div>
```

Event goes:

button → div → body → document

This is bubbling.

Stop using:

```
event.stopPropagation()
```

8. preventDefault() vs stopPropagation()

preventDefault()

Stops default action.

`event.preventDefault()`

Example:

- prevent form submit
 - prevent link opening
-

stopPropagation()

Stops bubbling.

9. Hoisting

var

Hoisted with undefined.

```
console.log(x)
var x = 5
```

Output:

undefined

let and const

Hoisted BUT in Temporal Dead Zone.

Access before declaration → error.

10. Closures (VERY IMPORTANT THEORY)

Function remembers parent variables.

```
function outer() {
  let x = 10;

  function inner() {
    console.log(x);
  }
}
```

```
    return inner;
  }
```

Used in:

- callbacks
- async
- React hooks

11. Callback Hell → Promises → Async/Await

Evolution asked often.

Callback

```
getData(function(){
  getMore(function(){
  });
});
```

Messy.

Promise

```
fetch()
.then()
.catch()
```

Cleaner.

Async/Await

Best readability.

```
async function test(){
  let data = await fetch(url);
}
```

12. this Keyword

VERY confusing topic.

In normal function

Depends on caller.

In arrow function

Does NOT create own this.

Uses parent this.

13. Arrow Function vs Normal Function

Arrow

```
const add = (a,b) => a+b;
```

Difference

- shorter
 - no own this
 - cannot be constructor
-

14. Local Storage vs Session Storage vs Cookies

localStorage

- permanent
- browser storage

sessionStorage

- deleted after tab close

cookies

- small data
 - sent with HTTP request
-

15. GET vs POST

GET

- fetch data
- visible in URL
- limited size

POST

- send data securely
 - used for forms/login
-

16. PUT vs PATCH

PUT

Replace full resource.

PATCH

Partial update.

17. CORS (Often asked)

Browser security feature.

Blocks requests from different origins.

Example:

frontend.com

calling

api.com

Need server permission headers.

18. HTTP vs HTTPS

HTTPS

HTTP + SSL/TLS encryption.

Secure.

Uses port 443.

19. REST API Basics

VERY important.

CRUD Mapping

Operation	HTTP Method
Create	POST
Read	GET
Update	PUT/PATCH
Delete	DELETE

20. Semantic HTML

Very likely theory question.

Use meaningful tags:

<header>

<nav>

<section>

<article>

<footer>

Benefits:

- SEO
 - accessibility
 - readability
-

21. Flexbox vs Grid

Flexbox

1D layout.

Row OR column.

Grid

2D layout.

Rows + columns.

22. Position in CSS

relative

Relative to itself.

absolute

Relative to nearest positioned parent.

fixed

Fixed to screen.

sticky

Sticky while scrolling.

23. display:none vs visibility:hidden

display:none

Removed from layout.

visibility:hidden

Hidden BUT space remains.

24. Debouncing vs Throttling

Advanced but VERY interview/exam favorite.

Debounce

Runs after user stops action.

Search box example.

Throttle

Runs every fixed interval.

Scroll example.

25. Virtual DOM (React)

React creates lightweight copy of DOM.

Updates only changed parts.

Faster performance.

26. Props vs State (React)

Props

Data from parent.

Read-only.

State

Internal component data.

Can change.

27. React Lifecycle / Hooks

Must know:

`useState()`

`useEffect()`

useRef()
useMemo()

useEffect

Handles:

- API calls
 - timers
 - side effects
-

28. SQL/Backend Things They RANDOMLY ASK in WPT

Sometimes out-of-syllabus.

Know basics:

Primary Key

Unique row identifier.

Foreign Key

Connects tables.

JOIN

Combines tables.

29. Authentication vs Authorization

Authentication

Who are you?

(login)

Authorization

What can you access?

(permission)

30. MOST IMPORTANT JS WEIRD OUTPUTS

Memorize these:

```
[] == false    // true
```

```
"" == false    // true
```

```
null == undefined // true
```

```
0 == false    // true
```

```
[] + []    // ""
```

```
[] + {}    // "[object Object]"
```

```
typeof null    // object
```

```
0.1 + 0.2
```

```
// 0.30000000000000004
```

Example:

```
console.log("A");
```

```
setTimeout(()=>{  
  console.log("B");  
},0);
```

```
console.log("C");
```

Output:

A

C

B

Why?

Because timeout callback waits in queue.

2. Call Stack

Functions go into stack.

Last in → first out.

Stack overflow happens with infinite recursion.

3. var vs let vs const

Feature	var	let	const
Scope	function	block	block
Redeclare	yes	no	no
Reassign	yes	yes	no
Hoisted	yes	yes	yes

MOST IMPORTANT:

- prefer const
 - use let if changing
 - avoid var
-

4. Shallow Copy vs Deep Copy

Shallow copy

Copies reference.

```
let a = {x:1};
```

```
let b = a;
```

Changing b changes a.

5. Spread Operator

VERY important ES6 feature.

```
let arr = [1,2];
```

```
let newArr = [...arr,3];
```

Output:

[1,2,3]

6. Destructuring

```
let [a,b] = [1,2];
```

Objects:

```
let {name} = user;
```

7. Template Literals

```
let name = "GK";
```

```
console.log(`Hello ${name}`);
```

Uses backticks.

8. Difference Between find() and filter()

find()

Returns FIRST match.

filter()

Returns ALL matches array.

9. map() vs forEach()

map

forEach

returns array

no return

used for transform

used for loop

10. some() vs every()

some()

At least ONE true.

every()

ALL must be true.

11. Synchronous vs Asynchronous

Synchronous

Line by line.

Asynchronous

Doesn't block execution.

Examples:

- API calls
 - timers
 - file reading
-

12. Promise States

pending

fulfilled

rejected

13. Promise Methods

.then()

.catch()

.finally()

14. Why React is Fast

Because:

- Virtual DOM
 - selective updates
 - component reuse
-

15. Controlled vs Uncontrolled Components

Controlled

React state controls input.

Uncontrolled

DOM controls input.

16. Keys in React

Used in lists.

Help React identify changed elements.

`<li key={id}>`

17. Middleware (possible theory)

Functions between:

request → response

Example:

- authentication
 - logging
-

18. JWT (possible)

JSON Web Token.

Used for authentication.

Contains:

- header
 - payload
 - signature
-

19. MVC Architecture

Model

View

Controller

Separates logic.

20. CDN

Content Delivery Network.

Faster loading using nearest server.

21. Responsive Design

Website adjusts to:

- mobile
- tablet
- desktop

Uses:

- media queries
 - flexbox
 - bootstrap grid
-

22. Bootstrap Grid

12-column system.

Example:

col-md-6

Means:

6/12 width on medium screens.

23. Media Query

@media(max-width:768px)

Used for responsiveness.

24. Accessibility (VERY POSSIBLE THEORY)

Make websites usable for everyone.

Use:

- alt text
 - labels
 - semantic tags
-

25. SEO Basics

Search Engine Optimization.

Improve ranking using:

- semantic HTML
 - meta tags
 - proper headings
-

26. Cookies vs Tokens

Cookies

Browser stores automatically.

Tokens

Manual auth mechanism.

27. Status Codes MUST KNOW

Code Meaning

200 OK

201 Created

301 Redirect

304 Not Modified

400 Bad Request

401 Unauthorized

403 Forbidden

404 Not Found

500 Server Error

28. MOST IMPORTANT OUTPUT LOGIC

If + sees string → concatenation.

$2 + "2" = "22"$

If - * / → number conversion.

$"5" - "2" = 3$

29. Weird Comparison Logic

$1 < 2 < 3$

becomes:

`true < 3`

`1 < 3`

`true`

But:

`3 > 2 > 1`

becomes:

`true > 1`

`1 > 1`

`false`

THIS is examiner favorite.